Weekly Report — Week 06

General Information

- **Group ID**: Group02
- **Project Name**: Personal Finance Manager
- **Course:** CS160-26A01
- **Date Range:** 07/11/2026 – 13/11/2026
- **GitHub Repository:** https://github.com/thvy2507/CS160-Group02-FinalProject

---

Team Members

Member	Name	Responsibility
TV1	Trần Hồng Vy	Team Leader / Core & Integration
TV2	Lê Thiện Khôi	Master Data & Wallets
TV3	Dương Thị Nhã Uyên	Transactions & Recurring
TV4	Trần Nhật Tân	Statistics & Dashboard

Tasks Completed This Week

> This reporting period focused on **Sprint 1 — Setup & Architecture**. The team implemented the core utilities, designed the shared data models and manager interfaces, prepared the Statistics/Dashboard skeletons, and merged all four feature Pull Requests into `main`.

1. Core Setup & Utilities — TV1

- Configured the C++17 project with CMake and an initial CLI menu through `FinanceManager`.
- Implemented `DynamicArray<T>` using manual dynamic memory management, including resizing, element access, removal, clearing, deep copying, and bounds checking.
- Implemented the `Date` class with current-date initialization, calendar validation, leap-year handling, and comparison operators.
- Added `tests/test_core.cpp` to test the Dynamic Array and Date utilities.
- Reviewed shared interfaces and coordinated integration with the other team members.
- Merged PR #6 — Core Utilities.

2. Master Data Models & Interfaces — TV2

- Defined the `Wallet`, `IncomeSource`, and `ExpenseCategory` models.
- Declared manager interfaces for CRUD operations, lookup by ID, and wallet balance management.
- Designed preliminary fixed-size binary records with Name Pool offsets and lengths, along with soft-delete flags.
- Standardized private member naming, English comments, and shared interface conventions during code review.

- Merged PR #7 — Master Data Models & Interfaces.

3. Transaction & Recurring Models — TV3

- Defined `Transaction` as an abstract base class, with `Income` and `Expense` derived models.
- Designed `RecurringTransaction` with a monthly schedule, optional end date, active status, and last-generated-date tracking to support duplicate prevention.
- Declared the `TransactionManager` and `RecurringTransactionManager` interfaces.
- Established links to Wallet, Income Source, and Expense Category managers through IDs and manager references.
- Documented ownership considerations for dynamically allocated transactions.
- Merged PR #8 — Transaction & Recurring Interfaces.

4. Statistics & Dashboard Design — TV4

- Defined interfaces for date-range summaries, wallet summaries, annual overviews, income by source, and expenses by category.
- Created `StatisticsManager`, `NumberFormatter`, `Dashboard`, and `StatisticsMenu` skeletons.
- Prepared example datasets, expected report results, design documentation, and a CLI smoke test.
- Updated Statistics interfaces to read transactions through `const TransactionManager&`, avoiding a `DynamicArray` of abstract `Transaction` objects.
- Added the four Statistics and UI `.cpp` files to the root `CMakeLists.txt`.
- Merged PR #5 — Statistics & Dashboard.

5. Shared Architecture & Integration

- Agreed on common data conventions: **`long long` for money in whole VND**, **`int` for object IDs**, and the shared **`Date` class**.
- Agreed that object IDs start at 1, are distinct from array indices, and are not reused after deletion.
- Reviewed and updated interfaces to maintain consistency between Core, Master Data, Transactions, and Statistics.
- Checked header compatibility across modules using C++17 compilation checks.
- Successfully compiled and linked the Statistics/Dashboard smoke test on macOS.
- Successfully built the integrated application with CMake on macOS **before the final PR merge**.
- Reviewed and merged all four Sprint 1 Pull Requests into `main` on 13/11/2026.

Implementation Status: Sprint 1 architecture, core utilities, module declarations, and integration skeletons have been merged. Core utility tests and selected compilation/smoke checks passed. Most CRUD operations, transaction processing, statistics calculations, formatted output, and binary persistence remain unimplemented. A fresh build of the final merged `main` on macOS and Windows is still to be verified.

AI Usage Declaration

The team declared the use of multiple AI tools in preparing the Sprint 1 architecture, code skeletons, tests, and documentation:

- **TV1 — Trần Hồng Vy:** ChatGPT (OpenAI) assisted with Core code review, shared types and interface alignment, PR review, Git troubleshooting, C++17 build/test commands, and documentation.
- **TV2 — Lê Thiện Khôi:** Gemini (Google) assisted with Master Data model design, fixed-size record and Name Pool

planning, manager interfaces, and CMake/Git guidance.

- **TV3 — Dương Thị Nhã Uyên:** Claude (Anthropic) assisted with Transaction and Recurring model declarations, manager interfaces, shared-type questions, and Git/GitHub workflow.
- **TV4 — Trần Nhật Tân:** ChatGPT (OpenAI) assisted with Statistics/Dashboard design, skeletons, sample test cases, and build/Git guidance. Additional AI usage in the current Week 06 reporting period has not been confirmed by TV4.

The members reviewed, edited, and committed their own work, while TV1 coordinated cross-module corrections and integration. AI assistance and passing skeleton tests do not mean the unfinished financial operations or persistence features are implemented.

Tasks Planned for Next Week

The team plans to begin **Sprint 2 — Core Management**, following `WEEKLY_PLAN.md`.

TV1 — Trần Hồng Vy (Core & Integration)

- Perform a fresh build and CLI smoke test from the latest merged `main`, including cross-platform verification where available.
- Connect the CLI menu to implemented module operations and improve input validation.
- Coordinate module integration, shared interfaces, and code reviews.
- Prepare tests for the first working end-to-end workflows.

TV2 — Khôi (Master Data & Wallets)

- Implement Wallet, Income Source, and Expense Category constructors and manager methods in `.cpp` files.
- Implement in-memory CRUD operations, ID assignment, and search by ID.
- Implement wallet balance operations and validate negative/invalid amounts.
- Test Master Data operations using the shared `DynamicArray`.

TV3 — Uyên (Transactions & Recurring)

- Implement Transaction, Income, Expense, and relevant manager methods.
- Implement in-memory income/expense creation and viewing, with input validation.
- Integrate transaction processing with TV2's wallet balance and ID lookup interfaces.
- Implement safe memory ownership and cleanup for transaction objects.

TV4 — Tân (Statistics & Dashboard)

- Implement basic date-range income, expense, and net balance calculations using real transaction data.
- Implement whole-VND number formatting with `long long`.
- Connect report data access to `TransactionManager` and test against the prepared sample cases.
- Continue integrating Statistics navigation and Dashboard display with `FinanceManager`.

Issues

Issue 1: Shared Types and Interface Inconsistencies

Initial module designs used different monetary types and transaction data access patterns. In particular, Statistics originally used `double` and a `DynamicArray<Transaction>` interface, which was incompatible with the agreed money type and abstract `Transaction` model.

How Addressed:

- Standardized money as `long long`, IDs as `int`, and dates through TV1's `Date` class.
- Revised Statistics interfaces to accept a read-only `TransactionManager` reference.
- Updated related code, tests, and documentation before integration.

Issue 2: Dependencies Between Feature Branches

Some feature branches were created before the shared Core utilities were merged, so their headers could not be compiled independently without the required dependencies.

How Addressed:

- Used temporary include copies and combined-header checks to validate interfaces.
- Integrated the latest `main` into the Statistics feature branch before updating the root CMake configuration.
- Verified that the integrated application compiled and linked on macOS before merging the final PR.

Issue 3: Incomplete Implementations and Test Coverage

The Sprint 1 deliverables intentionally emphasized models and interfaces. Most manager methods, statistics algorithms, and binary persistence features are not yet implemented, and header-only compilation does not prove runtime correctness.

How Addressed:

- Kept the scope of each Pull Request explicit in its description.
- Performed Core unit tests, header compilation checks, and Statistics/Dashboard smoke tests where applicable.
- Scheduled full CRUD, transaction processing, reporting logic, and integration tests for subsequent sprints.

Issue 4: Integration and Cross-Platform Verification

Changes were reviewed and merged into `main`, but a fresh build of the final merged branch and Windows verification have not yet been confirmed.

How Addressed:

- Added Statistics and Dashboard source files to the root CMake configuration and verified an integrated macOS build before merge.
- Planned a clean build and smoke test from the final `main` branch as the next verification step.
- Retained the Friday demo checkpoint for team-wide validation and reporting.